Функции, указатели и составные типы

SLICE, MAP И STRUCT
НА ПРИМЕРЕ МОДЕЛИ ЗАКАЗОВ



X5 Tech

Какую задачу решает эта тема?

ГЛАВНАЯ ЦЕЛЬ

Соберать небольшую модель заказов из функций и составных типов

ЧТО ДАЁТ НАВЫК

Выносить логику в функции

Передавать параметры и возвращать неск. значений

Работать с указателями

Использовать var, :=, const и zero values

Моделировать данные

Использовать struct, slice и map по назначению

Отлаживать код

Ставить breakpoint и отслеживать изменение состояния

Поговорим о том, что уже прошли

1. Умеем выбирать базовые типы
2. Умеем писать if и switch
3. Умеем обходить данные циклом
4. Умеем запускать автотесты

СЛЕДУЮЩИЙ ШАГ

Разобрать длинный main на функции и понятные типы



Модель заказа

Опишем заказ через struct, позиции через slice, статусы через map и изменим заказ по указателю.

STRUCT → ФУНКЦИИ → МЕТОДЫ

2 МИНУТЫ НА ОБСУЖДЕНИЕ

Как функция возвращает результат и ошибку?

Предложите сигнатуру функции создания заказа, если входные данные могут быть неверными.

Функция возвращает 2 результата

Идиома value, error делает неуспех частью обычного потока

ВЫЗОВ:

Результат:	order
Ошибка:	err
Проверка	if err != nil

ИСХОДНЫЙ КОД:

```go
func NewOrder(id int) (Order, error) {
    if id <= 0 {
        return Order{}, errors.New("bad id")
    }
    return Order{ID: id}, nil
}
```

Значение или указатель

Go всегда передаёт значение;
указатель тоже копируется как адрес

ИСХОДНЫЙ КОД:

```
func Rename(o Order) {
    o.Name = "copy"
}

func RenamePtr(o *Order) {
    o.Name = "saved"
}
```

ЧТО ИЗМЕНИТСЯ СНАРУЖИ:

1. Order создаёт копию структуры
2. *Order передаёт копию адреса
3. Изменение через * сохраняется



Указатель выбирают ради общей изменяемой сущности

Когда изменение теряется

Передача структуры по значению
меняет только локальную копию



ЧТО ИДЁТ НЕ ТАК:

- ✖ Поле изменилось только внутри функции
- ✖ nil-указатель разыменовали
- ✖ Большая структура копируется зря
- ✖ Вызов скрывает изменение состояния

КЛЮЧЕВОЙ ВЫВОД

Указатель нужен, когда функция должна изменить общий объект. В остальных случаях достаточно значения

Выбираем семантику

Составные типы решают разные задачи

Struct, slice и map дополняют друг друга,
но ведут себя по-разному

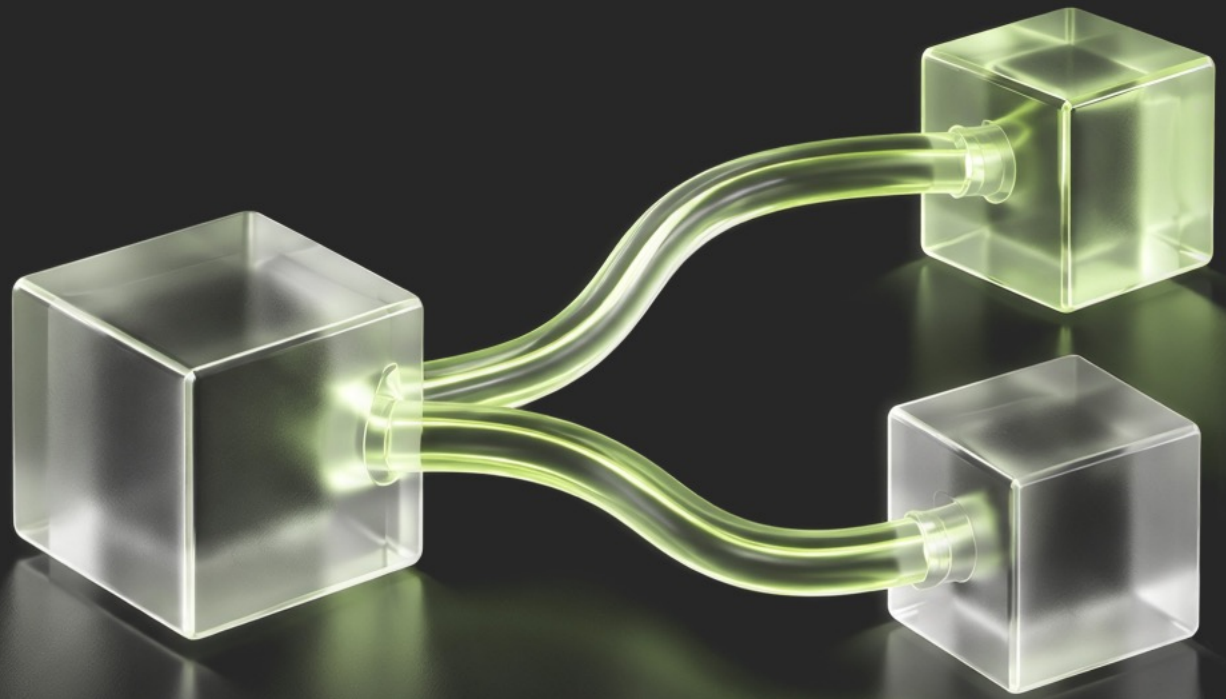
STRUCT:

- Набор именованных полей
- Копируется как значение
- Методы описывают поведение
- Литерал создаёт экземпляр

SLICE И MAP:

- **slice:** последовательность
- **map:** ключ и значение
- slice может делить общий массив
- map не хранит порядок ключей

Разделяйте работу на шаги и явно управляйте изменением



Функция

Задаёт контракт

Указатель

Показывает, что операция может изменить переданный объект

Что хранит модель заказа

Struct объединяет поля, slice хранит позиции, map помогает со справочниками

ОПРЕДЕЛЕНИЕ

«Order» – это…

Структура, которая хранит идентификатор, позиции и статус заказа.

struct · slice · map

КАК СОБРАТЬ МОДЕЛЬ:

1. Описать Item и Order
2. Создать заказ функцией
3. Добавлять позиции по указателю

Первая модель заказа

ИСХОДНЫЙ КОД:

```
type Item struct {
    Name  string
    Price int64
}

type Order struct {
    ID     int
    Items  []Item
    Status string
}
```

ОБЪЯСНЕНИЕ

Сейчас важна структура данных

- Поля структуры отражают язык предметной области
- Имена полей и типы должны отражать ограничения задачи

Функции над заказом

Разделим создание, изменение
и вычисление

1 **CREATE**
Создаём валидный заказ

`NewOrder(id)`

2 **MUTATE**
Добавляем позицию

`AddItem(&order, item)`

3 **CALCULATE**
Считаем сумму

`Total(order)`

Разбираем изменение по шагам

ИСХОДНЫЙ КОД:

```
func AddItem(o *Order, item Item) error {
    if o == nil {
        return errors.New("nil order")
    }
    o.Items = append(o.Items, item)
    return nil
}
```

ОБЪЯСНЕНИЕ

Указатель позволяет функции изменить исходный заказ

- ***Order** – адрес изменяемого заказа
- **nil** – защита от паники
- **append** – добавление в slice
- **error** – сообщение о неуспехе

Каждая строка поддерживает контракт функции.

slice: len, cap и append

ИСХОДНЫЙ КОД:

```
// terminal

items := make([]int, 0, 2)
items = append(items, 10)
items = append(items, 20)

fmt.Println(len(items) // 2)
fmt.Println(cap(items) // 2)
items = append(items, 30)
```

ОБЪЯСНЕНИЕ

append может переиспользовать общий массив или создать новый

- Создаём slice → добавляем элементы
- Смотрим на len и cap → ищем разницу

Когда map равен nil

ИСХОДНЫЙ КОД:

```
var statuses map[int]string

fmt.Println(statuses[1]) // ""

statuses[1] = "new" // panic: assignment to entry
```

Что ожидали:

Добаление статуса

Что получили:

Panic

Как исправить

make(map[int]string)



map нужно создать перед первой записью

Модель собирается из ролей

Каждый тип и функция отвечают
за одну часть задачи

ИЗ ЧЕГО СОСТОИТ МОДЕЛЬ ЗАКАЗА:

1. STRUCT данных
2. Функции поведения

Slice хранит позиции

map помогает со справочниками

Указатель сохраняет изменение

Понятные роли уменьшают случайные связи.

Путь модели заказа

От отдельных типов
к проверяемому поведению

1. описать Item
2. описать Order
3. создать NewOrder
4. вернуть error
5. добавить через *Order
6. посчитать Total
7. запустить отладчик



Сейчас соберём эту модель и проверим её тестами

ЗАДАНИЕ ДЛЯ ПРАКТИКИ

Соберите мини-модель заказов и пройдите её отладчиком

1. Описать struct Item и Order
2. Написать NewOrder с error
3. Добавлять Item через *Order
4. Посчитать сумму позиций
5. Проверить код breakpoint и тестами

Рефлексия



Удалось ли:

- вернуть два значения из функции
- Понять разницу между & и *
- Различить роль slice и map
- Защититься от nil
- Увидеть изменения в отладчике

Где это пригодится в работе?

Что осталось непонятным?

Домашнее задание

Задание:	Реализовать функции согласно сигнатурам, обработать nil и пустые входы, добиться прохождения автотестов
Формат сдачи:	Ссылка на ветку в репозитории
Критерии оценивания:	Оценка согласно проценту пройденных автотестов (более подробно в репозитории)



ТЕМА СЛЕДУЮЩЕГО ЗАНЯТИЯ:

Интерфейсы

+ Разбор задания



Итоги занятия

1. Функции и составные типы превращают код в понятную модель
2. Понятная модель показывает, что можно сделать с данными и где ждать ошибку

ТЕПЕРЬ ВЫ УМЕЕТЕ:

1. Разделять логику на функции
1. Возвращать value и error
1. Моделировать данные через struct
1. Работать со slice и map

X5 Tech